

LAB 2 – LAYOUT VIEW

2.1. GIỚI THIỆU

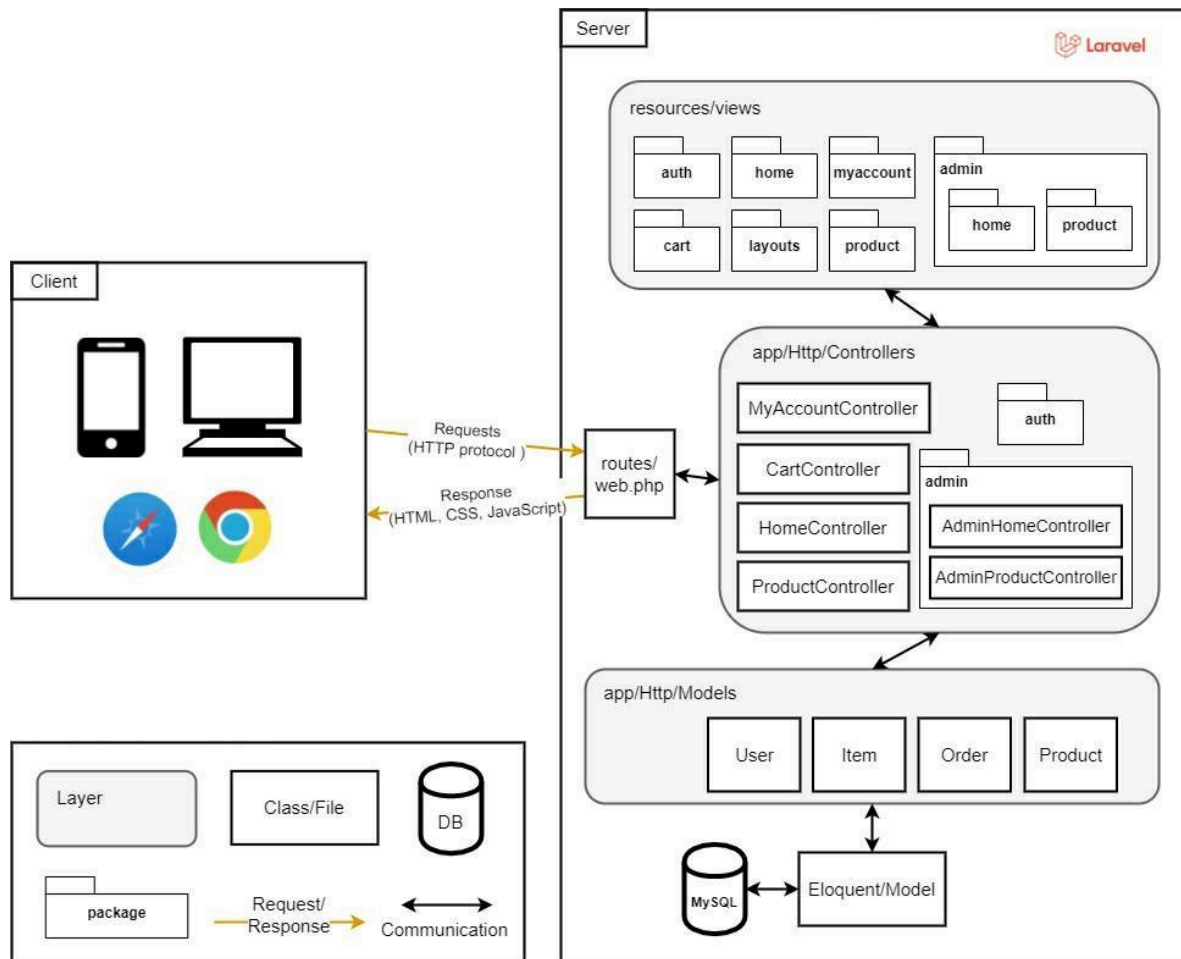
Có nhiều cách khác nhau để thiết kế và triển khai các ứng dụng web. Ví dụ: có thể tạo toàn bộ ứng dụng web bằng cách đặt code vào một tập tin duy nhất. Tuy nhiên, việc tìm ra lỗi trong một tập tin như vậy (chứa hàng nghìn dòng code) không phải là một việc dễ dàng. Các cách tiếp cận khác chia code trên các tập tin và thư mục khác nhau. Thậm chí sẽ tìm thấy các phương pháp phân chia ứng dụng thành các ứng dụng nhỏ khác nhau được phân phối trên nhiều máy chủ (việc phân phối các máy chủ này không phải là một nhiệm vụ dễ dàng).

Như có thể thấy, việc cấu trúc code không phải là một nhiệm vụ dễ dàng. Đó là lý do tại sao các nhà phát triển và nhà khoa học máy tính đã phát triển cái được gọi là các mẫu kiến trúc phần mềm (architectural patterns). Các mẫu kiến trúc phần mềm là các bố cục cấu trúc được sử dụng để giải quyết các vấn đề thiết kế phần mềm thường gặp phải. Với những mẫu này, những người bắt đầu và nhà phát triển mới vào nghề không cần phải “phát minh lại” mỗi khi họ bắt đầu một dự án mới. Có nhiều mẫu kiến trúc, chẳng hạn như mô model-view-controller, layers, hướng dịch vụ và dịch vụ vi mô (micro-services). Mỗi cái đều có ưu điểm và nhược điểm, được nhiều người áp dụng rộng rãi. Tuy nhiên, một trong những mẫu được sử dụng nhiều nhất là mẫu model-view-controller (MVC).

Model-view-controller (MVC) là một mẫu kiến trúc phần mềm thường được sử dụng để phát triển các ứng dụng web có giao diện người dùng. Mẫu này chia ứng dụng thành ba phần được kết nối với nhau.

1. **Model** chứa logic nghiệp vụ của ứng dụng. Ví dụ: dữ liệu sản phẩm của ứng dụng Cửa hàng trực tuyến và các chức năng của nó.
2. **View** chứa giao diện người dùng của ứng dụng. Ví dụ: view để đăng ký sản phẩm hoặc người dùng.
3. **Controller** hoạt động như một giao diện giữa các phần model và view. Ví dụ: controllers sản phẩm thu thập thông tin từ view "tạo sản phẩm" và chuyển nó đến model sản phẩm để lưu trữ trong cơ sở dữ liệu.

Mẫu MVC cung cấp một số ưu điểm: phân tách code tốt hơn, nhiều thành viên trong nhóm có thể làm việc và cộng tác đồng thời, tìm lỗi dễ dàng hơn và khả năng bảo trì được cải thiện. Hình 2-1 thể hiện kiến trúc phần mềm Cửa hàng trực tuyến mà chúng ta sẽ triển khai trong hướng dẫn này.



Hình 2-1: Online Store software architecture.

Hãy cùng phân tích nhanh về kiến trúc này:

- Ở bên trái, chúng ta có clients (người dùng ứng dụng, ví dụ: trình duyệt trên thiết bị di động/máy tính). Clients kết nối tới ứng dụng thông qua HTTP (Hypertext Transfer Protocol). HTTP cung cấp cho người dùng cách tương tác với ứng dụng web.
- Ở bên phải, chúng ta có máy chủ, nơi đặt code ứng dụng.
- Tất cả các tương tác của máy khách trước tiên đều được chuyển cho một tập tin tuyến có tên là web.php
- Tập tin web.php chuyển tương tác tới controllers.
- Controllers giao tiếp với các model và truyền thông tin đến các views, cuối cùng được gửi đến client dưới dạng code HTML, CSS và JavaScript.

Chúng ta đánh dấu các lớp Model, View và Controller bằng màu xám. Chúng ta có bốn model (thực thể) tương ứng với các lớp được xác định trong sơ đồ lớp (trong Hình 1-5). Như đã đề cập, có nhiều cách tiếp cận khác nhau để triển khai ứng dụng web với Laravel. Thậm chí còn có nhiều phiên bản MVC khác nhau được sử dụng trong ứng dụng Laravel.

2.2. LAYOUT VIEW

2.2.1. GIỚI THIỆU BLADE

Blade là một công cụ tạo khuôn mẫu mạnh mẽ được tích hợp trong Laravel. Các tập tin mẫu Blade sử dụng phần mở rộng tập tin là `.blade.php` và thường được lưu trữ trong thư mục `resources/views`. Trong các tập tin blade, sẽ có sự kết hợp giữa code HTML với các chỉ thị Blade và các phần tử Blade. Lệnh Blade là các phím tắt thuận tiện cho các cấu trúc điều khiển PHP phổ biến, chẳng hạn như các câu lệnh và vòng lặp có điều kiện.

Ví dụ: đoạn code sau hiển thị một đoạn trích của một views đơn giản trong Laravel bằng cách sử dụng PHP đơn giản.

Code

```
<?php if($records > 0) { ?>
    I have records!
<?php } else { ?>
    I don't have any records!
<?php } ?>
```

Tương tự, nhưng với các chỉ thị của Blade, sẽ như sau:

Code

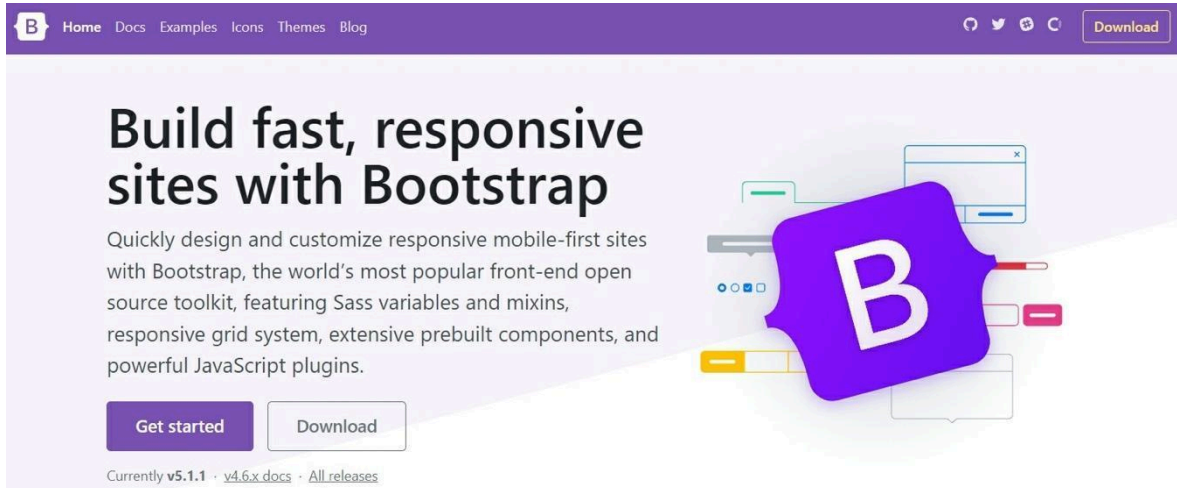
```
@if (count($records) > 0)
    I have records!
@else
    I don't have any records!
@endif
```

Không cần công cụ tạo khuôn mẫu trong các dự án PHP. Có thể kết hợp code logic ứng dụng (PHP) với code xem ứng dụng (HTML) trong bất kỳ tập tin PHP nào. Tuy nhiên, sự kết hợp đa ngôn ngữ này không được hỗ trợ cho các ngôn ngữ khác như Java hoặc Python. Vậy tại sao các framework PHP lại sử dụng các công cụ tạo khuôn mẫu? Lý do chính là để tránh sử dụng cú pháp PHP hoặc thẻ PHP bên trong tập tin view. Thay vào đó, nên sử dụng các chỉ thị hoặc trợ giúp (helpers) của công cụ tạo khuôn mẫu. Lợi thế là gì? Công cụ tạo mẫu giới hạn số lượng chức năng có sẵn được triển khai trong view (để cung cấp sự phân tách code thích hợp). Ví dụ: với PHP và HTML, có thể tạo kết nối cơ sở dữ liệu hoặc thậm chí một lớp PHP bên trong tập tin xem (điều này thật điên rồ vì các views không chịu trách nhiệm tạo các kết nối hoặc lớp cơ sở dữ liệu). Vì vậy, công cụ tạo mẫu đảm bảo rằng sẽ không tạo ra những điều điên rồ trong view. Khuyến nghị là: nếu không tìm thấy lệnh hoặc trợ giúp cho chức năng cần triển khai trong tập tin view, thì đó là do chức năng đó không nên được triển khai trong view (có thể nó nên được triển khai trong controllers hoặc trong một trình điều khiển khác).

Không sử dụng code PHP đơn giản trong view. Blade cho phép điều đó, nhưng đừng làm vậy. Blade chứa lệnh `@php` cho phép chèn code PHP đơn giản. Tuy nhiên, chỉ sử dụng nó như là phương án cuối cùng. Chúng ta đã phát triển các ứng dụng web Laravel phức tạp mà không cần sử dụng lệnh đó.

2.2.2. GIỚI THIỆU BOOTSTRAP

Bootstrap là khung CSS phổ biến nhất để phát triển các trang web đáp ứng và ưu tiên thiết bị di động (xem Hình 5-1). Đối với các dự án Laravel, nhà phát triển có thể thiết kế giao diện người dùng từ đầu nếu muốn. Nhưng vì hướng dẫn này không nói về giao diện người dùng nên chúng ta sẽ tận dụng các khung CSS (chẳng hạn như Bootstrap) và sử dụng mẫu để tạo ra thứ gì đó trông chuyên nghiệp. Có thể tìm hiểu thêm về Bootstrap tại: <https://getbootstrap.com/>.



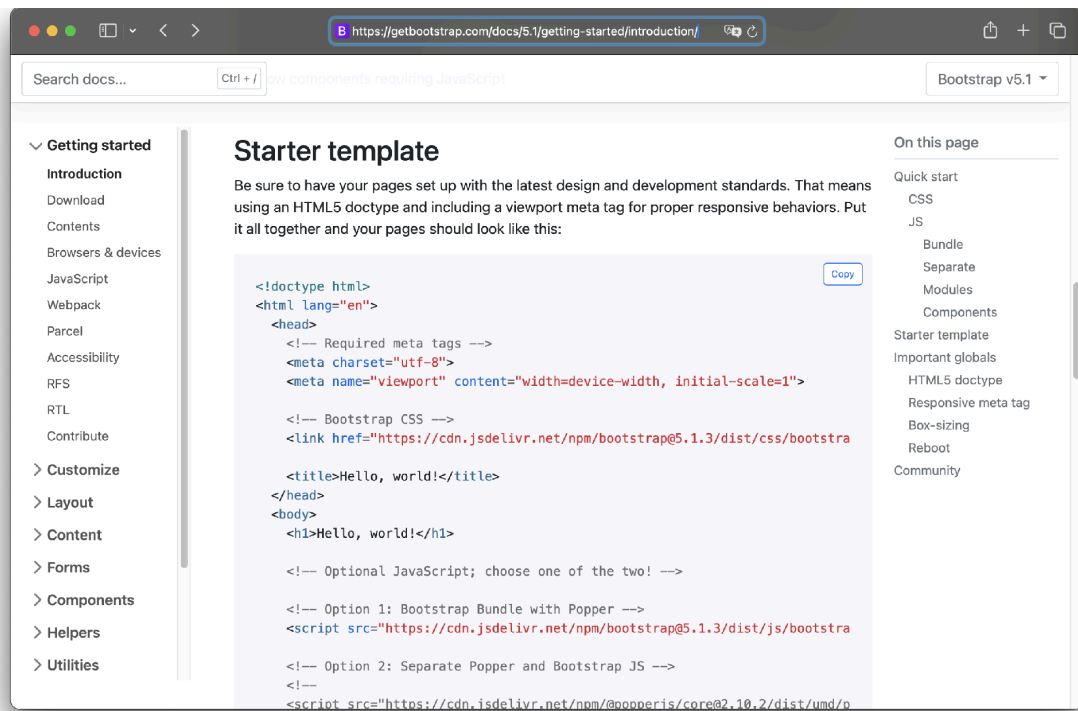
Hình 2-2: Bootstrap website.

2.2.3. GIỚI THIỆU BLADE LAYOUTS

Hầu hết các ứng dụng web đều duy trì bố cục chung giống nhau trên nhiều trang khác nhau (tiêu đề chung, thanh điều hướng và chân trang). Tuy nhiên, việc duy trì ứng dụng sẽ vô cùng phức tạp nếu chúng ta phải lặp lại toàn bộ HTML đầu trang, thanh điều hướng và chân trang trong mọi view. May mắn thay, chúng ta có thể định nghĩa bố cục này dưới dạng một tập tin Blade duy nhất và sử dụng nó trong toàn bộ ứng dụng.

Tạo app.blade.php

Để bắt đầu với Bootstrap và bố cục Blade, trước tiên chúng ta tạo một thư mục có tên là layouts trong thư mục resources/views. Sau đó, chúng ta sử dụng mẫu Bootstrap để tạo bố cục (xem Hình 2-3). Có thể tìm thấy mẫu Bootstrap tại: <https://getbootstrap.com/docs/5.1/getting-started/introduction/>.



Hình 2-3. Bootstrap starter template.

Trong resources/views/layouts, tạo một tập tin mới app.blade.php và thêm code sau vào.

Code

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css"
rel="stylesheet" crossorigin="anonymous" />
    <title>@yield('title', 'Online Store')</title>
</head>
<body>
    <!-- header -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-secondary py-4">
        <div class="container">
            <a class="navbar-brand" href="#">Online Store</a>
            <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarNavAltMarkup" aria-controls="navbarNavAltMarkup"
aria-expanded="false" aria-label="Toggle navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNavAltMarkup">
                <div class="navbar-nav ms-auto">
                    <a class="nav-link active" href="#">Home</a>
                    <a class="nav-link active" href="#">About</a>
```

```
        </div>
    </div>
</div>
</nav>
<header class="masthead bg-primary text-white text-center py-4">
    <div class="container d-flex align-items-center flex-column">
        <h2>@yield('subtitle', 'Online Store - Laravel Framework')</h2>
    </div>
</header>
<!-- header -->
<div class="container my-4">
    @yield('content')
</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js"
crossorigin="anonymous">
</script>
</body>
</html>
```

Đoạn code trên dựa trên code mẫu Bootstrap và Thanh điều hướng Bootstrap (<https://getbootstrap.com/docs/5.1/components/navbar/>). Chúng ta đã sửa đổi code cơ sở, bao gồm các liên kết trong tiêu đề (Home và About). Mẫu bao gồm tập tin CSS Bootstrap (bootstrap.min.css) và tập tin Bootstrap JS (bootstrap.bundle.min.js). Và đã bao gồm ba chỉ thị @yield Blade.

@yield được sử dụng làm điểm đánh dấu. Chúng ta sẽ chèn code vào các điểm đánh dấu đó từ các Blade views con (sử dụng lệnh @section). @yield sử dụng hai tham số, tham số đầu tiên là code định danh điểm đánh dấu. Giá trị thứ hai là giá trị mặc định sẽ được thêm vào nếu view con không thêm code cho điểm đánh dấu đó.

Chỉnh sửa welcome.blade.php

Xóa tất cả code hiện có trong resources/views/welcome.blade.php và thêm code sau vào.

Code

```
@extends('layouts.app')
@section('title', 'Trang chủ - Online Store')
@section('content')
<div class="text-center">
    Chào mừng đến với ứng dụng Online Store!
</div>
@endsection
```

View welcome mở rộng view layouts.app. View này đưa thông báo 'Trang chủ - Online Store' vào @yield('title') của layouts.app và đưa một div HTML với thông báo chào mừng bên trong @yield('content') của layouts.app.

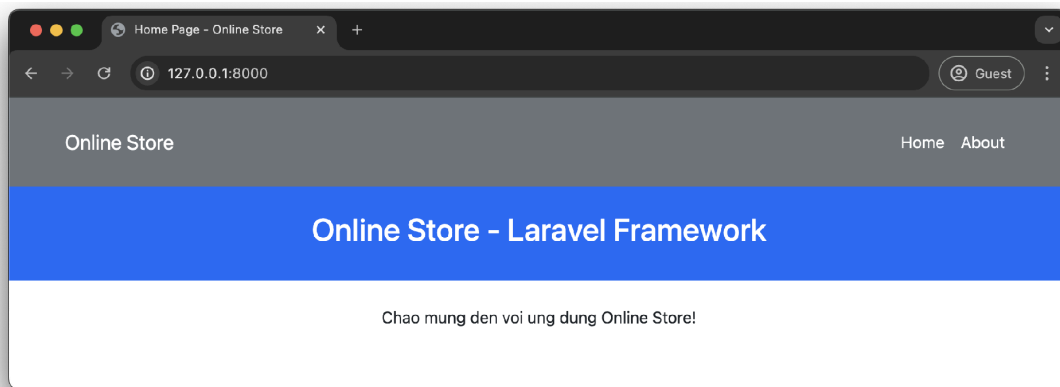
Chạy ứng dụng

Trong Terminal, đi tới thư mục dự án và thực hiện như sau:

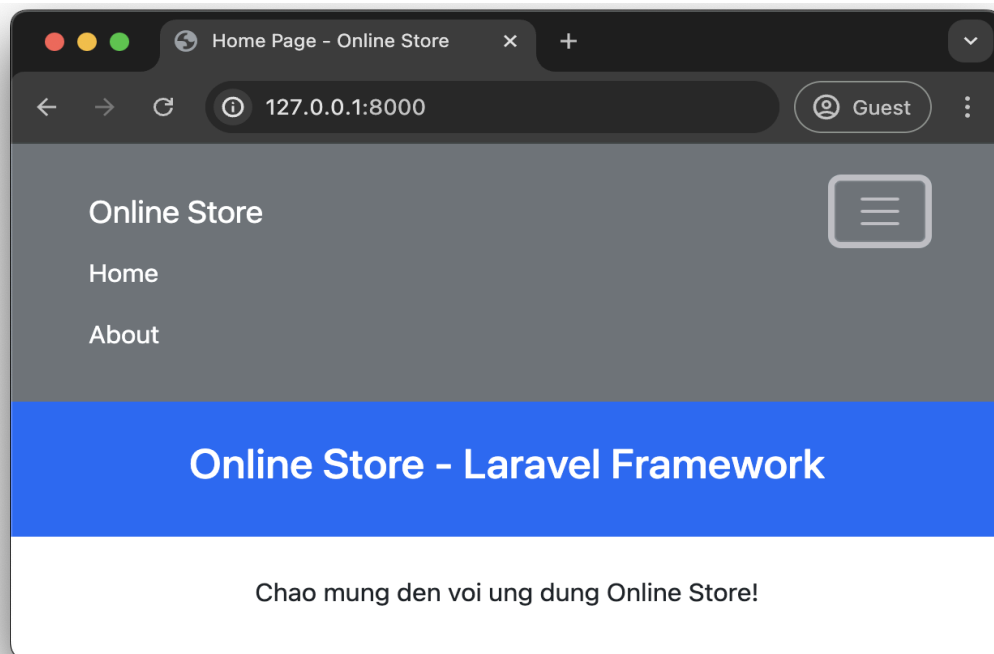
Terminal

```
php artisan serve
```

Mở trình duyệt đến <http://127.0.0.1:8000/> và sẽ thấy ứng dụng có bố cục mới (xem Hình 2-4). Nếu giảm độ rộng cửa sổ trình duyệt, sẽ thấy một thanh điều hướng phản hồi nhanh (nhờ mẫu bootstrap, thanh điều hướng và bao gồm các tập tin bootstrap, xem Hình 2-5).



Hình 2-4: Trang chủ ứng dụng với layout.



Hình 2-5. Trang chủ ứng dụng có độ rộng cửa sổ trình duyệt giảm.

2.2.4. THÊM TÙY CHỈNH CSS STYLES VÀ FOOTER

Để giao diện ứng dụng chuyên nghiệp hơn. Chúng ta sẽ bao gồm tập tin CSS tùy chỉnh và chân trang trong bố cục.

Kiểu tùy chỉnh (app.css)

Tạo một thư mục css trong thư mục public/. Sau đó, trong public/css, tạo một tập tin mới app.css và điền thông tin sau vào tập tin đó.

Code

```
.bg-secondary {
    background-color: #2c3e50 !important;
}
.copyright {
    background-color: #1a252f;
}
.bg-primary {
    background-color: #1abc9c !important;
}
nav{
    font-weight: 700;
}
.img-card{
    height: 18vw;
    object-fit: cover;
}
```

Chúng ta có một số kiểu CSS tùy chỉnh trong tập tin trên. Chúng ta ghi đè một số phân tử Bootstrap bằng các giá trị và màu sắc.

Sửa đổi app.blade.php

Cuối cùng, trong resources/views/layouts/app.blade.php, hãy các thay đổi in đậm (màu đỏ) sau để bao gồm tập tin CSS trước đó và tạo phần chân trang.

Code

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css"
rel="stylesheet" crossorigin="anonymous" />
    <link href="{{ asset('/css/app.css') }}" rel="stylesheet" />
    <title>@yield('title', 'Online Store')</title>
</head>
<body>
    <!-- header -->
```



```

<nav class="navbar navbar-expand-lg navbar-dark bg-secondary py-4">
  <div class="container">
    <a class="navbar-brand" href="#">Online Store</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarNavAltMarkup" aria-controls="navbarNavAltMarkup"
aria-expanded="false" aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNavAltMarkup">
      <div class="navbar-nav ms-auto">
        <a class="nav-link active" href="#">Home</a>
        <a class="nav-link active" href="#">About</a>
      </div>
    </div>
  </div>
</nav>
<header class="masthead bg-primary text-white text-center py-4">
  <div class="container d-flex align-items-center flex-column">
    <h2>@yield('subtitle', 'Online Store - Laravel Framework')</h2>
  </div>
</header>
<!-- header -->
<div class="container my-4">
  @yield('content')
</div>

<!-- footer -->
<div class="copyright py-4 text-center text-white">
  <div class="container">
    <small> Copyright - <a class="text-reset fw-bold text-decoration-none"
target="_blank" href="https://twitter.com/">
      NĐDuy
    </a> - <b>CKC</b>
    </small>
  </div>
</div>
<!-- footer -->

  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js"
crossorigin="anonymous">
  </script>
</body>
</html>

```

Laravel bao gồm nhiều hàm global helper PHP. Ví dụ: hàm asset tạo URL cho nội dung bằng cách sử dụng cơ chế hiện tại của request (HTTP hoặc HTTPS). Vì tập tin css/app.css của chúng ta nằm trong thư mục public nên nó sẽ được tự động triển khai qua liên kết máy chủ (tức là `http://127.0.0.1:8000/css/app.css`). Chúng ta đã sử dụng dấu ngoặc nhọn `{{ }}` để gọi hàm asset. Dấu ngoặc nhọn được sử dụng trong các tập tin Blade để hiển thị dữ liệu được truyền đến view hoặc gọi trợ giúp Laravel helpers. Cuối cùng, chúng ta đã tạo phần chân trang có tên tác giả và liên kết tới tài khoản Twitter.

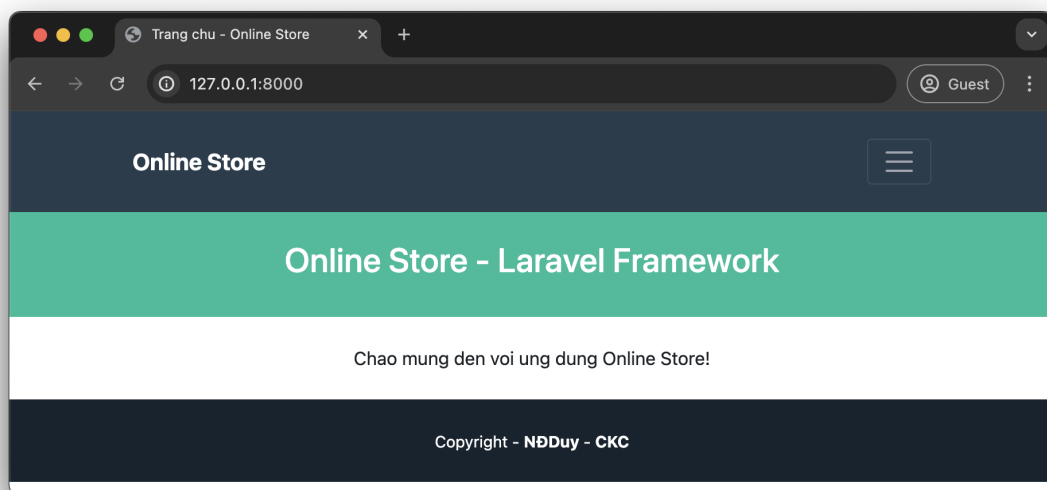
Chạy ứng dụng

Trong Terminal, đi tới thư mục dự án và thực hiện như sau:

Terminal

```
php artisan serve
```

Sẽ thấy trang chủ của chúng ta với bố cục thay đổi (xem Hình 2-6). Nó trông chuyên nghiệp hơn. Cái này thiết kế được lấy cảm hứng từ mẫu Bootstrap miễn phí có tên Freelancer. Có thể tìm thêm thông tin về mẫu này tại đây: <https://startbootstrap.com/theme/freelancer>.



Hình 2-6. Trang chủ ứng dụng với bố cục tinh tế hơn.